

Ex. No. : 09

Date:

Register No.: 231701016

Name: Harish A

A PYTHON PROGRAM TO IMPLEMENT KNN AND K-MEANS

Aim:

To implement **KNN and K-Means algorithms** on the credit card dataset to perform classification and clustering of transactions.

Requirements:

Jupyter Notebook with Python, creditcard.csv dataset, and required Python libraries.

PROGRAM:

Step 1: Import Required Libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
from sklearn.preprocessing import StandardScaler
```

Step 2: Upload Dataset

```
from google.colab import files
uploaded = files.upload()
```

Upload:

creditcard.csv

Step 3: Load the Dataset

```
data = pd.read_csv("creditcard.csv")

data.head()
```

...	Time	V1	V2	V3	V4	V5	V6	V7
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941

5 rows × 31 columns

Step 4: Define Features and Target

```
X = data.drop("Class", axis=1)
y = data["Class"]
```

Step 5: Train-test split

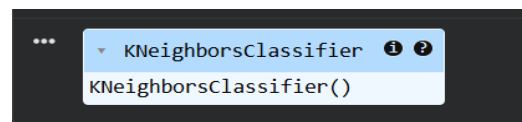
```
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y
)
```

Step 6: Create KNN model

```
knn = KNeighborsClassifier(n_neighbors=5)
```

Step 7: Train model

```
knn.fit(X_train, y_train)
```



Step 8: Predict the Results

```
y_pred = knn.predict(X_test)
```

Step 9: Evaluate the Model

Accuracy

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Confusion Matrix

```
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

Classification Report

```
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

```
... KNN Accuracy: 0.9995435553526912

Classification Report:
              precision    recall  f1-score   support

     0       1.00      1.00      1.00     56864
     1       0.92      0.81      0.86         98

 accuracy          0.96
 macro avg          0.96
 weighted avg       1.00

Confusion Matrix:
[[56857   7]
 [   19  79]]
```

Step 9: Import libraries for K-Means

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
```

Step 10: Apply K-Means (2 clusters: Fraud & Normal)

```
kmeans = KMeans(n_clusters=2, random_state=42)
kmeans.fit(X_scaled)
```

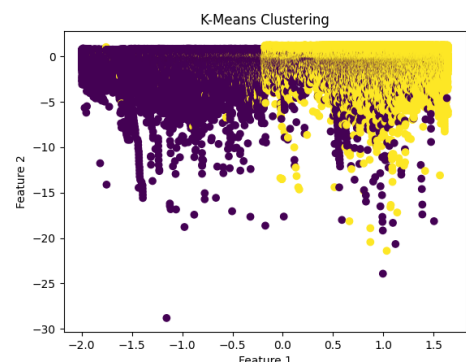
Step 11: Cluster labels and add to dataset

```
clusters = kmeans.labels_
df['Cluster'] = clusters
print(pd.crosstab(df['Class'], df['Cluster']))
```

Cluster	0	1
Class		
0	155038	129277
1	299	193

Step 12: Visualization (using 2 features for plotting)

```
plt.scatter(X_scaled[:,0], X_scaled[:,1], c=clusters)
plt.title("K-Means Clustering")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```



Result:

Thus, KNN successfully classified transactions, and K-Means grouped them into clusters, helping identify patterns in fraudulent and normal transactions.